

Assignment 2

COL775: Deep Learning

Semester II, 2025–2026

Due Date: May 7, 2026

Notes:

- You should use **Python=3.10** and **Pytorch=2.5.0+cu11.8** for all your programming solutions.
 - You should submit all your code as well as any graphs that you might plot. Submit all the code in .py files and all answers to theory questions; all graphs should be submitted in a PDF. Do not submit any theory in the code files. Do not submit the datasets.
 - Include a **single write-up (pdf) file** which includes a brief description for each question explaining what you did. Include any observations and/or plots required by the question in this single write-up file.
 - Your code should have appropriate documentation for readability. Demo will also be conducted in which you will have to explain your code. You will be graded based on what you have submitted as well as your ability to explain your code.
 - The readability and presentation in your report will also carry marks. While there is no specific format, you are advised to be clear, concise and complete in your answers with explanations wherever required.
 - A substantial portion of your marks will be based on the accuracy achieved, so ensure that your models are trained thoroughly and properly.
 - Your assignment will be run on autograder scripts. Detailed instructions regarding this and submission guidelines of code will follow soon. Till then, please start working on writing the functions for your code.
 - **This assignment can be done in groups of 2 or 3 students. Use this [google form](#) to fill out your team details**
 - Refer to the [course website](#) for assignment submission instructions.
 - We plan to run Moss on the submissions. We will also include submissions from previous years since some of the questions may be repeated. Any cheating will result in a zero on the assignment, an additional penalty upto the negative of the total weightage of the assignment and possibly much stricter penalties (including a **fail grade** and/or referring to a **DisCo**). Please note that using any **AI tools is not allowed**.
-

Overview

In this assignment you'll build a comprehensive understanding of modern representation learning by exploring three complementary paradigms: weakly-supervised contrastive learning, self-supervised learning and generative modelling. All experiments will be conducted on a version of CLEVR dataset [6], along with programmatically generated captions. Data is available [here](#).

In the first part (A/Aa), you'll implement from scratch two influential works in this space : CLIP [9] and DINO [1]. CLIP leverages weak supervision through image-text contrastive learning, while DINO relies on self-supervised knowledge distillation to learn visual features without labels. You will train both models and analyze the learned representation through [linear probing](#) on downstream tasks such as object counting, color/shape prediction etc.

In the second part (B), you will extend these learned visual representations to build a general purpose vision-language model (VLM). This involves integrating a pre-trained language model with the vision encoder and aligning the two modalities in a two-stage training pipeline inspired by LLaVA [8]. By freezing the vision encoder and training a projection interface, you will study how effectively the learned representations support language grounding and reasoning on CLEVR.

In part (C), you will implement a Variational Auto-Encoder (VAE) to learn a dense latent-representation on this dataset. Using which you'll train a text-guided Latent Diffusion Model (LDM) to generate diverse CLEVR scene from any given caption.

1 (A) Representation Learning

In this section, you will implement and train two separate models. Your goal is to learn a high-dimensional embedding space where images with similar semantic concepts (e.g., object counts, colors, shapes) are clustered together, without using explicit classification labels. Only use the data inside **Part_A** directory for this part.

1.1 Contrastive Language-Image Pre-training (CLIP)

You will implement a weakly-supervised model inspired by CLIP. You will use the programmatically generated captions provided in the datasets to align image and text modalities. We only provide some basic hyper-parameter details, however you should read the paper carefully to understand other detailed nuances.

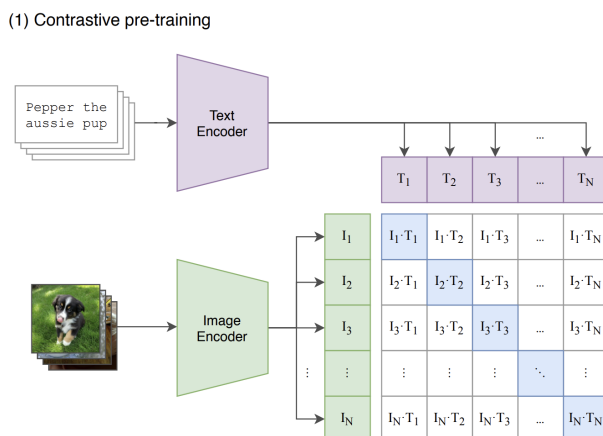


Figure 1: CLIP framework, Source: [9]

As shown in 1, the model consists of an image encoder, a text encoder, and a shared embedding space in which aligned image-text pairs are brought close together using a contrastive objective.

Overall model: Given an image I and its caption T , the model produces an image embedding $v \in \mathbb{R}^d$ and a text embedding $t \in \mathbb{R}^d$. Both embeddings are ℓ_2 -normalized and compared using temperature-scaled cosine similarity. The temperature is a learnable parameter and is initialized to $\log(1/0.07)$, following the details described in the CLIP paper [9]

Vision Transformer image encoder: The visual pipeline should follow the standard ViT design [2]. The image encoder is a Vision Transformer consisting of 12 transformer layers, 6 attention heads per layer, hidden dimension 384 and MLP dimension 1536. In the experiments use the patch size of 16 and the input image resolution is fixed at 224×224 . You must project your final output into a 512 dimensional shared space. Each transformer layer can be implemented via Pytorch abstractions like `nn.MultiheadAttention`, `nn.Linear` etc. Use of fully abstracted Transformer and TransformerLayer is not allowed.

Text encoder: The text branch is a lightweight transformer. It should be implemented with 6 transformer layers, 6 attention heads, 384 hidden dimension and 1536 MLP dimension. Refer to [10] for details. After the transformer stack, the representation of the final non-padding token (effectively the end-of-text token) should be projected into the joint embedding space (512 dimensional).

Tokenizer and Caption representation: Instead of using a pretrained tokenizer, You should construct a simple task-specific tokenizer directly from the training captions. You can choose a fixed context length for training the model. Each caption is formed directly from the scene graph and encodes object count and attributes such as size, color, material, and shape. A typical caption has the form:

An image with 3 objects: 1 small red metal cube, 2 large blue rubber spheres

Since, no spatial information is used in the captions, therefore many different images can share the same caption. If several identical captions appear in one contrastive batch, the learning signal becomes less clean. You must take this into account while constructing your batches.

Objective Function and Optimization Training uses the standard symmetric CLIP objective described in the paper. You can follow these parameters as default, however feel free to experiment on the best setting that works for you (You can refer to the paper for additional values of hyper-params): Optimizer=AdamW, learning rate= 5×10^{-4} , weight decay= 0.2, $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-6}$, epochs: 100, LR Scheduler: Cosine Decay with Linear warmup.

1.2 Self-Supervised Distillation - DINO

You will implement a self-supervised model inspired by DINO. Unlike CLIP, this model does not use text at all. It learns visual representations by enforcing consistency between different augmented views of the same image. Similar to CLIP, we only provide basic hyper-parameter details, you should carefully read the [1] for detailed instructions.

Overall framework: The model consists of two networks with identical architecture:

- a **student** network, which is updated by gradient descent,
- a **teacher** network, not updated by backpropagation; instead, its parameters are updated as an exponential moving average of the student parameters.

Both the Teacher and Student models must use the same Vision Transformer backbone as used in CLIP (12 layers, 6 heads and 384 hidden dimension). On top of the ViT backbone, both models use a projection head of 4096 dimension which maps the backbone feature into the space used for self-distillation. (Note that this is different from the paper which uses a 65536 dimensional feature space). Feel free to figure out what works best here as well.

Multi-crop data augmentation: A central component of DINO is the augmentation pipeline. For each image, the transform produces: (1) two **global crops** of size 224×224 , (2) several **local crops** of size 96×96 . You can use the default number of local crops as 8, so each image yields 10 views in total. Refer to the paper for more details on the different augmentation strategies applicable.

Teacher–student training rule: During training, the student receives all augmented views and the teacher receives only the two global views. The teacher is not updated by backpropagation. Instead, its parameters are updated as an exponential moving average of the student parameters as specified in the paper. The objective function is cross-entropy between the teacher probability distribution for one global crop and the student prediction for every other crop of the same image, excluding the matching same-view pair. The final loss averages this over all valid teacher–student crop pairs. Refer to Equation 3 in [1].

Centering mechanism. A key stabilization component is the running *center* which is an exponential moving average of the teacher outputs across batches. Before applying softmax to the teacher outputs, the current center is subtracted. Refer to Equation 4 and Algorithm 1 in [1]. This prevents the teacher distribution from becoming too peaky and helps avoid representation collapse.

Optimization. Training is performed only on the student network. The optimizer to be used is AdamW with same default hyper-parameters as CLIP. You’re free to ablate your hyper-parameter choices.

2 (Aa) Representation Analysis

Once training is complete, you will **freeze the weights** of all Vision Encoders (from both CLIP and DINO). The goal of this section is to rigorously evaluate whether the learned representations capture meaningful semantic concepts such as object count, color composition, and shape composition. You will perform both **quantitative** and **qualitative** analyses to probe the structure and utility of the learned embedding space. Only use the data inside **Part_Aa** directory for this part.

2.1 Linear Probing

Linear probing is a standard protocol for evaluating representation quality by measuring how easily downstream tasks can be solved using a simple linear classifier on frozen features. We’ll use two tasks, (1) Object Count Prediction and (2) Color Set Prediction, you’ll find the corresponding .json files and images in the dataset. Extract the fixed-dimensional image embeddings from the frozen Vision-Encoders and train a single linear layer classifier (no hidden layers or activation) on top of them.

- Use appropriate loss functions i.e Cross-Entropy for Counting task and Binary-Cross Entropy (multi-label) for shape prediction task.
- Ensure that only linear classifier’s weights are updated during training.
- Report the classification accuracy for Counting Task and **F1 score** for Color-Prediction Task, specifically fill the values in Table 1.
- Provide brief analysis discussing:
 - Describe you results, which model performs better on which tasks?
 - Which model encodes semantic attributes most effectively?

Model	[CLS] Embedding				GAP over Patches			
	Counting		Color Pred.		Counting		Color Pred.	
	Train	Val	Train	Val	Train	Val	Train	Val
CLIP								
DINO Student								
DINO Teacher								

Table 1: Representation analysis across models and embedding strategies. GAP refers to Global-Average-Pooling operation, which averages the output embeddings of all N image patches (excluding [CLS] token), resulting in a single D -dimensional vector.

2.2 t-SNE Visualization

To complement quantitative evaluation, you will perform a qualitative analysis of the embedding space using **t-SNE** (**t**-distributed **S**tochastic **N**eighbor **E**mbedding). Use the extracted embeddings of training set (70K images) and project them into a 2D space. Generate scatter plots for CLIP and both DINO models (Student and Teacher), color coding each point based on the object count in the image to visualize the global structure. Analyze and comment your observations.

2.3 Cross-Modal Retrieval using CLIP

To further evaluate the semantic consistency of the learned representations in CLIP, you will perform an **retrieval** task based on nearest neighbors in the embedding space.

For each image in the **validation** split, compute its embedding and retrieve the embedded captions from a pool consisting of all validation captions. Both image and text embeddings are L2-normalized, and similarity is computed using cosine similarity. Report **Recall@K** (**R@1**, **R@3**), which measures whether the ground-truth caption is among the top-K retrieved results. Additionally evaluate text→image retrieval by using each caption as a query and retrieving images from the validation set. Report the **same** (**R@1** and **R@3**) metrics.

Show a few queries alongside their top retrieved neighbors for both (Image→Text and Text→Image retrieval). Comment on success and failure cases for both.

3 (B) Vision-Language Modeling

In this part of assignment, you'll use the best performing trained vision-encoder from part (A) and align it's learned representation with a pre-trained LLM to unlock QA and general task capabilities on CLEVR dataset.

The vision-encoder must be loaded from the checkpoint and it's representation (all the patch-tokens of an image) should pass through a 2-layer MLP with reverse-bottleneck. The output dimension of the MLP must match with the hidden representation size of the LLM. We'll use **Qwen/Qwen3-4B-Instruct-2507** as the LLM backbone and follow the guidelines proposed by [8] which involves a two-stage training process.

- **Stage-1** corresponds to core-alignment stage where both the Vision-encoder and LLM weights are kept frozen and just the MLP projector's weights are trained on Image-Captioning task.
- **Stage-2** instantiates the best model from previous stage and involves finetuning both the MLP projector and the LLM backbone simultaneously (vision-encoder is still frozen) on diverse questing-answering dataset.

3.1 Implementation

- **Stage-1** Use the captions for for `Clevr_official` split available [here](#) to train this stage.
- **Stage-2** You should apply LoRA (rank $r = 16$) [5] matrices inside attention block of the LLM for fewer trainable parameters and small memory footprint. You can use `peft` package that allows for easy LoRA instantiation. You will train this model on chain-of-thought style question-answering i.e given a question, the model is trained to predict it's reasoning and then the answer. Use the QA dataset inside `Clevr_official` directory with explanations available [here](#), which contains multiple `factual_explanation` per QA pair.

Training stage-2 can be memory intensive, you should explore techniques like [gradient-checkpointing](#), [gradient-accumulation](#) and mixed-precision training in PyTorch.

3.2 Evaluation

Report exact-match accuracy for stage-2 and stage-1, additionally report bleu score (via `sacrebleu` package) for stage-1 as well, on train and validation sets. Visualize correct and incorrect samples from validation set and report your findings.

4 (C) Generative modeling via Latent Diffusion Models

In this part, we'll learn a text-conditional Latent-Diffusion Model (LDM) to generate 3D-like compositional scenes from CLEVR dataset. The pipeline consists of two models that must be trained sequentially:

- **A Convolutional Variational Autoencoder (VAE)** : Trained to compress RGB images into a compact continuous latent space. This allows to greatly reduce the compute required for training.
- **A Conditional U-Net (LDM)** : A diffusion model trained to denoise the learned latents, conditioned on spatially-agonistic text embeddings from a frozen-text encoder (CLIP text-encoder).

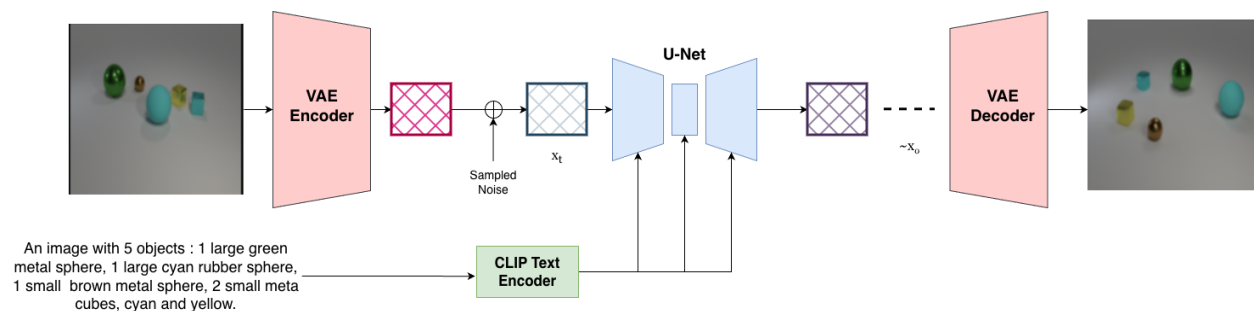


Figure 2: Overall training pipeline for Part C.

4.1 Implementation Details

Only use the dataset provided in the `Part_A` directory. To reduce the computational load, resize the images to (128x128) resolution. A detailed visualization for both the architectures is available [here](#).

4.2 VAE

The VAE is trained to compresses a (128x128x3) image into a (16x16x4) representation.

- **Encoder** : Convolve input to 32 channels. Progressively downsample using `nn.Conv2d` (stride 2) through 3 stages (32ch, 64ch, 128ch). Each stage contains 2 ResBlocks.
- **Bottleneck (Mid)**: 2 ResBlocks. Convolve the 128ch feature map to 8 channels.
- **Latent Projection**: Split the 8 channels into Mean (4ch) and LogVar (4ch). Apply the reparameterization trick to sample the 16x16x4 latent z .
- **Decoder**: Convolve the latent to 128 channels. Passes through 2 Mid ResBlocks. Progressively upsamples (using `nn.Upsample(scale_factor=2, mode='nearest')`) followed by a 3x3 Conv) through 3 stages (128ch, 64ch, 32ch). Each stage contains 2 ResBlocks. Finally, projects to 3 RGB channels with a Tanh activation.

The loss as described in Eq.10 in [7] is the sum of MSE loss between input and decoded image, and the scaled KL-loss computed analytically where σ and μ are predicted by the model. Train this model to at least 100 epochs using [SiLU](#) activation and Group Normalization as needed. You can keep $\lambda = 1e-6$ (D_{KL} term scaling factor).

4.3 LDM/U-Net

The conditional U-net takes three inputs:

- **Noisy Latent** (z_t) : 16x16x4
- **Timestep embedding** (t) : Encoded via Sinusoidal embeddings (similar to transformer) and MLPs into a time vector, which is added to the ResBlocks.
- **Text-Embeddings** : Encoded via frozen CLIP-text encoder ([openai/clip-vit-base-patch32](#)) into contextual embeddings (`batch`, `seq_len`, 512)

Architecturally, it consists of three stages:

- **DownStages**: Uses Downsample Convs (stride=2). Incorporates Spatial Transformers (Self/Cross-Attention) at the specified attention levels. Self-attention refers to the attention computed where vae-latents forms the query, keys and values. In Cross-attention, vae-latents form the queries whereas text representation creates the keys and values.
- **MidStage**: Consist of a SpatialTransformer block with a ResBlock on both sides.
- **UpStages**: Concatenates skip connections from DownStages. Uses Nearest Upsampling + Conv. Incorporates Spatial Transformers at the specified attention levels.

Important Tips:

- The VAE latents do not automatically follow a zero-mean, unit-variance distribution. You must compute the mean and standard deviation of the un-noised VAE latents across the training set, and then shift and scale the latents using these statistics before adding diffusion noise.
- During training, to enable prompt guidance at inference, randomly replace the text context with a learned empty/null embedding 10% of the time. Read more on this in [4]
- Sample a timestep t , add noise ϵ to the normalized latent to obtain z_t . Pass z_t , t , and the text context to the U-Net. The loss is the mean squared error (MSE) between the predicted noise $\hat{\epsilon}$ and the true noise ϵ .
- **Hyperparameters** : Number of attention heads = 8, Number of diffusion timesteps = 500, noising schedule = cosine, epochs ≥ 100 .

4.3.1 Inference and Evaluation

During inference:

1. Start with a pure Gaussian noise tensor of shape (batch, 4, 16, 16).
2. Encode your target prompt using the CLIP text encoder to get the context embedding.
3. Iteratively denoise for 500 steps using the DDPM sampling equations.(Line 4, Algorithm 2 in [3])
4. At each step, apply Classifier-Free Guidance (CFG). Run the U-Net twice: once with the text context ($\epsilon_{\theta}(z_t, c)$) and once with the learned null context ($\epsilon_{\theta}(z_t, c = \phi)$). Combine them (Eq. 6 in [4]):

$$\hat{\epsilon}_{\theta}(z_t, c) = (1 + s) \cdot \epsilon_{\theta}(z_t, c) - s \cdot \epsilon_{\theta}(z_t) \quad (1)$$

(Recommended guidance scale $s = 4.0$.)

5. After the final timestep, **de-normalize** the predicted latent using the pre-computed training statistics.
6. Pass the de-normalized latent through the frozen VAE Decoder to obtain the final (128x128x3) RGB image.

Evaluation For Both VAE and LDM, report the Frechet Inception Distance (FID) metric that evaluates the quality and diversity of generated images. Reconstruct (incase of VAE) and generate (via captions in case LDM) all 10K validation images and compute FID using [clean-fid](#) package. Visualize your reconstructed and generated samples and report your observations.

Submission Guidelines:

Please submit your fully documented code, the trained model weights, and a final PDF report containing your plots, metric tables, and a brief discussion comparing the two paradigms. Detailed guidelines will be released later.

References

- [1] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers, 2021.
- [2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [3] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [4] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.
- [5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [6] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C. Lawrence Zitnick, and Ross Girshick. Clevr: A diagnostic dataset for compositional language and elementary visual reasoning, 2016.
- [7] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [8] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning, 2023.
- [9] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.